Design And Analysis of Algorithms
Lecture 8: Linear Programming (For Matching).

February 16, 2018

In this lecture, we introduce to concept of linear programming, with an eye towards solving matching problems. In general, it's useful to think of linear programming as a tool, and ask whether it's possible to capture enough structure of a problem such that solving the LP is computationally inexpensive, and the solution to the LP is a good one for the problem we started out with.

We'll now introduce some basic notation and definitions to work with linear programs.

Definition 1. *Define $a \in \mathbb{R}^n, b \in R$. Then a* half-space *of $\mathbb{R}^n$ is*

$$\{\vec{x} \in \mathbb{R}^n | \vec{a}x \geq b\}$$

Definition 2. *A* polyhedron *is the intersection of finitely many half-spaces.*

Note that a polyhedron is convex, as it is the intersection of (finitely many) convex sets. Use the notation $K = \{x : Ax \leq b\}$ to refer to some polyhedron for $A \in \mathbb{R}^{m \times n}, \tilde{\mathrm{A}}\mathring{\mathrm{U}} \in R^n, b \in R^m$, respectively.

Definition 3. *A* polytope *is a bounded polyhedron, that is, there is some $r \in \mathbb{R}_{\geq 0}$ such that $K \subseteq B(0, r)$.*

A *linear program* in n dimensions is defined by a polyhedron K and an n-dimensional vector c :

$$\min \sum_{i=1}^{n} c_i x is wbjectto \overrightarrow{x} \in K.$$

Notice that we need not extend this definition to include maximization problems, as we can simply flip the sign of c and solve a minimization.

Given a polytope K, a point $x \in K$ is

1. *extreme* if there aren't two $x_1 \neq x_2 \in K$ and some $\lambda \in (0, 1)$ such that $x = \lambda x_1 + (1 - \lambda)x_2$
2. a *vertex* if there exists some $d \in \mathbb{R}^n$ such that $dx < dy$ for all other $y \in K$
3. a *basic feasible solution* if it satisfies n linearly independent constraints defining K with equality.

One might suspect a relationship between these definitions: given a polytope, they are equivalent.

Fact 1. *Given a polytope K, 12 and 3 are equivalent.*

A useful property of LPs is that an optimal solution can always be found amongst the extreme points, or vertices, of the polytope defining the feasible region.

Fact 2. *For an LP (K, c) , where K is a polytope, there is an optimal solution which is extreme.*

This suggests the following algorithm for finding an optimal solution to an LP: compute and compare the objective value at all the vertices of K; there are at most $mchoosen$ of them (n of the m constraints are tight at a bfs).There are actually polynomial-time algorithms for solving LPs, so this isn't considered a very efficient algorithm, though the simplex method (which we'll discuss in a later lecture) does a smarter version of this, starting at some vertex and moving to a neighboring vertex; for most rules determining which neighboring vertex is "next", we know that this method can run in exponential time.

Given $X = \{x_1, \ldots, x_N\} \subset \mathbb{R}^n$, the *convex hull* of X is defined as

$$CH\left(X\right) = \{x \in \mathbb{R}^n | \exists \lambda_1, \ldots, \lambda_N \geq 0, \sum_{i=1}^{N} \lambda_i = 1, \sum \lambda_i x_i = x\}$$

We note that a convex hull is a polytope, and given a a polytope K, $CH(\{x \in \mathbb{R}^n : x \text{ is extreme for } K) = K$.

1 LPs for Perfect Matchings (in Bipartite Graphs)

Recall the bipartite matching setting; there is some graph $G = (L, R, E)$. Denote matchings (and other subsets of E) as vectors belonging to $\{0,1\}^{|E|}$. Then, the $|E|$-dimensional polytope defined by taking the convex hull of all perfect matchings is just

$$C_{pm} = CH\left(X \in \{0,1\}^{1E1} : X \text{ is a perfect matching in } G\right)$$

If one wants to find a minimum cost perfect matching, one can then define the objective coefficients c to be the weights of all the edges. So, using C_{pm} as our feasible region and c as our objective, we have defined a linear program. Can we solve it efficiently? The vertices of the feasible region are all perfect matchings, and so the LP is integral (there are no vertices that represent fractional matchings), but C_{pm} is "large": the number of perfect matchings can be of exponential size (think about the complete bipartite graph on $2n$ vertices), so this description will have exponentially many vertices and constraints.

This leads us to try a different formulation. Define the convex set

$$K_{PM} = x \in \mathbb{R}^{|E|} \text{ s.t } \begin{cases} \sum_{r \in R} x_{\ell r} = 1 & \forall \ell \in L \\ \sum_{\ell \in L} x_{\ell r} = 1 & \forall r \in R \\ x_{\ell r} \geq 0 & \forall \ell \in L, r \in R \end{cases}$$

We claim that, despite this LP having only linearly many constraints, it defines the same feasible space as the convex hull of all perfect matchings.

Theorem 1. $K_{PM} = CH(PM)$.

Proof. Any perfect matching satisfies the linear constraints of K_{PM}; it has exactly one edge incident to each vertex and has non-negative weightings on the edges, and therefore so will any convex combination of perfect matchings, so $CH(PM) \subseteq K_{PM}$.

To show $K_{PM} \subseteq CH(PM)$, we will consider all extreme points in K_{PM}; it will suffice to show they all belong to $CH(PM)$ (since if any extreme point belongs, so will any convex combination, since both sets are convex). Fix any extreme point in K_{PM}. Let $supp(x) \subseteq [|E|]$ denote the set of dimensions on which x has nonzero weight.

Claim 1. *The subgraph on those edges in $supp(x)$ is acyclic.*

Suppose there were a cycle in $supp(x)$; it must be of even length since G is bipartite. Let $x_1, \ldots, x_{2\ell|}$ be the support of x which has a cycle. Then, since each vertex in this cycle has two adjacent edges with strictly positive weight, $x_i \in (0,1)$ for all $i \in 2\ell$ (otherwise we'd have a violation of the constraint on weight at most one adjacent to any vertex). Since there are finitely many points which are strictly between 0 and 1, there is

some $\epsilon \in (0,1)$ such that $x_i+\epsilon < 1, x_i-\epsilon > 0$ for all i. Then, consider $\bar{x} = (x_1-\epsilon, x_2+\epsilon, \ldots, x_{2\ell-1}-\epsilon, x_{2\ell}+\epsilon)$ and $\underline{x} = (x_1+\epsilon, x_2-\epsilon, \ldots, x_{2\ell-1}+\epsilon, x_{2\ell}-\epsilon)$; then $\frac{1}{2}(\bar{x}+\underline{x} = x$, and both $\bar{x}, \underline{x} \in K_{PM}$ by construction. But this is a violation of the definition of x being extreme, so we have a contradiction and therefore the support of x cannot have a cycle.

This implies that $supp(x)$ has a leaf v with a edge $(v,z) = e$ with weight 1. Then neither v nor z can have any other adjacent edges in $supp(x)$; thus after removing that edge from G one can repeat the argument about there not being any cycles in $supp(x)$ to find another leaf; thus x must be integral and therefore $x \in CH(PM)$. □

1.1 Min-cost (non-perfect) matchings on bipartite graphs

If we move to the setting where we want to compute non-perfect matchings, how should we change our LP formulation? This problem isn't trivial if one allows for negative edge weights; if there's a negative edge in the graph then some non-empty matching will be of strictly lower cost than the empty matching.

We relax the feasible region from above to allow for adjacent edges to sum to at most 1:

$$K_M = x \in \mathbb{R}^{|E|} \text{ s.t } \begin{cases} \sum_{r \in R} x_{\ell r} \leq 1 & \forall \ell \in L \\ \sum_{\ell \in L} x_{\ell r} \leq 1 & \forall r \in R \\ x_{\ell r} \geq 0 & \forall \ell \in L, r \in R \end{cases}$$

It is a good exercise to show that this set is equal to the convex hull of all matchings on G; the proof is similar to the one above.

1.2 Arborescences and MSTs

Recall the definition of an r-arborescence in a directed graph: one can reach the "root" r via a directed path from each other $v \in V$, the out-degree of all non-r vertices should be 1, and the out-degree of r should be 0. If we want to formulate the min-weight r-arborescence problem as an LP, we can write down

$$K_{Arb} = \in \mathbb{R}^{|A|} \text{ s.t. } \begin{cases} \sum_{n \neq v} x_{v,n} = 1 & \forall v \neq r \\ \sum_{i \in S, j \notin S} x_{ij} \geq 1 & \forall S \subset V, r \notin S \\ x_{ij} \geq 0 & \forall i,j \in V \end{cases}$$

The same techniques as before would allow one to show that this set is equal to the convex hull on arborescences of G.

1.3 MST LPs

A simple LP to define the spanning tree feasible region is similar in some ways to that of an arborescence; there must always be an edge "leaving" any set, there cannot be too many edges "inside" a set, and there must be $|V|-1$ edges overall. These together will ensure connectivity and no additional edges.

$$K_{ST} = \in \mathbb{R}^{|E|} \text{ s.t. } \begin{cases} \sum_{v \in S, w \notin S} x_{v,v'} \geq 1 & \forall S \subsetneq V, S \neq \emptyset \\ \sum_{i,j \in S} x_{ij} \leq |S|-1 & \forall S \subset V, S \neq \emptyset \\ \sum_{i,j \in V} x_{ij} = |V|-1 x_{ij} \geq 0 & \forall i,j \in V \end{cases}$$

Again similar techniques can be used to show this actually captures the convex hull of all spanning trees for a graph G.

Remark 1. *Both of the previous LP formulations were "big"; their set of constraints was exponentially sized in n. However, these formulations can still be solved in polynomial time by use of something called a "separation oracle" which, given point x tells you whether x belongs to the feasible region or returns a violated constraint. Both of these LPs have efficient separation oracles, and those can be used with the Ellipsoid algorithm to solve these LPs efficiently. See* `http://theory.epfl.ch/osven/courses/Approx13/Notes/lecture9and10.pdf` *for details on their separation oracles.*

2 Perfect matchings in general graphs

If one moves to general rather than bipartite graphs, and looks at a straightforward LP formulation, for example this one:

$$K_{pm} = x \in \mathbb{R}^m \text{ s.t. } \begin{cases} \sum_{w \neq v} x_{v,w} = 1 & \forall v \in V \\ x_{v,w} \geq 0 & \forall v, w \in V \end{cases}$$

there are additional complications. Notice that odd cycles can cause us problems (our argument about acyclicity of the support of an extreme solution no longer goes through); in fact, consider a triangle: the vector $(\frac{1}{2}, \frac{1}{2}, \frac{1}{2})$ is a feasible solution to the above LP but is not in the convex hull of matchings for the graph G. So, the above LP has solutions which are not convex combinations of matchings, and therefore has some extreme points which are not matchings.

So, we need a more constrained LP for this problem. The first one you might try would be to get rid of the issue of odd cycles, by forcing odd cycles to have an edge leaving them (since, for a perfect matching, an odd number of vertices must have an edge to some other vertex). This can reformulated as the following set of constraints:

$$\{ \sum_{i \in S, j \notin S} x_{i,j} \geq 1 \quad \forall S : |S| \text{ is odd } \} = ODD$$

which sort of seems like a hack to "fix" the previous issue. Somewhat miraculously, this "hack" fixes everything! Define K_{genPM} to be the region intersection the set of constraints ODD along with those for K_{PM}

Theorem 2 (Edmonds [1965])**.**

$$CH(GPM) = K_{genPM}$$

This LP is "big" as mentioned about the previous two LPs; it's actually known one needs exponentially many constraints to describe this problem. But, as before, there is still an efficient way to solve this LP, it just takes some care.

The sketch of the proof takes a bfs in K, and looks at the linearly independent tight constraints. If there are any integral dimensions, drop them and recur. If no integral dimension exists, then every vertex had degree at least 2. If all of them have degree 2, one a has a cycle of nonzero weight, and one can play the trick we played for bipartite matchings to cause a contradiction. Otherwise one has a vertex of degree at least 3; then one can find a set on which some ODD constraint is tight; contract that set and argue recursively.

References

Jack Edmonds. Paths, trees, and flowers. *Canadian Journal of mathematics*, 17(3):449–467, 1965.